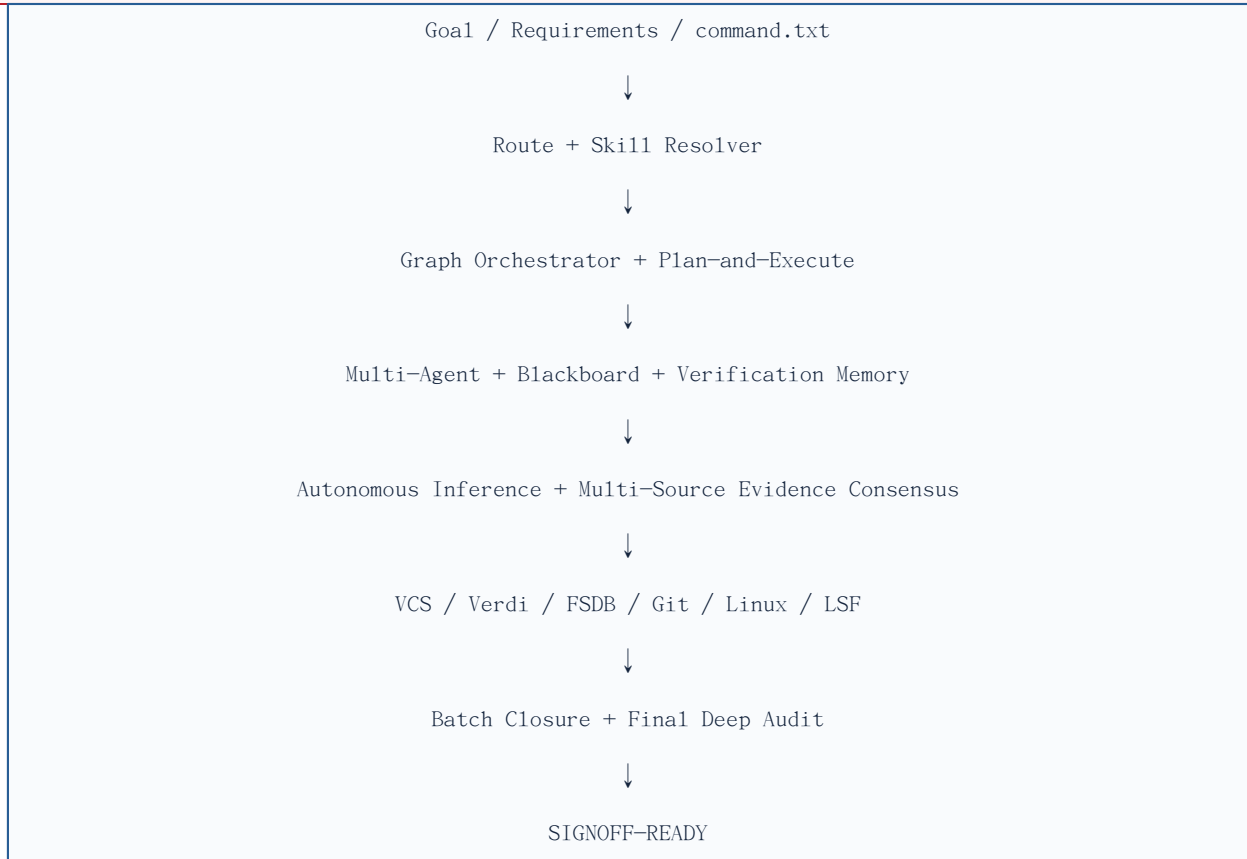


DV Agent Harness L5

詳細使用手冊 | 自主驗證 Multi-Agent Verification Memory Autonomous Inference Evidence
Closure

面向一般 DV / DE Engineer 的 Generic Verification Agent Harness：從 Block/IP、Subsystem、Multi-Subsystem 到 Full SoC，以 Graph、Plan、Multi-Agent、Evidence、Memory、Inference、VCS/Verdi/FSDB、Git/DevOps 與 LSF Regression 建立可控制、可追蹤、可重複的驗證閉環。



手冊內容

- DV Agent Harness 的觀念、用途與 AI 架構。
- Graph-Orchestrated、Plan-and-Execute、Multi-Agent、Blackboard、Evidence-Grounded ReAct。
- 7 Primary Agents、Dynamic Per-Job Agents 與 Skill-Governed Execution。
- Autonomous Inference：Hypothesis、Counter Evidence、Evidence Gap、Next-Best-Action。
- Multi-Agent 平行查 DUT RTL、PHY 文件、Programming Guide/Register、VIP examples/manual/source/class reference。
- Verification Memory：做過一次、驗證成功、下次能利用經驗，但 Memory 不取代 Current Evidence。
- Generic UVM、branch-A / interrupt-driven branch_fw / branch-B scenarios。
- WAVE / PA / FSDB_START / FSDB_STOP / TIMEOUT、FSDB/fsdbreport/Verdi debug。
- LSF One Job = One Agent、Change-only Alert、Periodic Snapshot、Exact-job Early Kill。
- Git/DevOps、Regression fixed SHA、Batch Closure、Final Deep Audit、Human Control。

1. DV Agent Harness 是什麼？

- 它不是單一聊天機器人，也不是把 shell command 包在 AI 外面；它是 Verification Orchestration Layer。
- 工程目標會被轉成 Graph、Plan、Agent Task、Evidence、Finding、Regression Batch 與 Closure State。
- 核心原則是「AI 可以自主執行，但重要判斷必須有 Evidence、Counter Evidence、SHA 與 Human Override」。
- 「L5」是此 Harness 自訂的 Verification Autonomy 分級名稱，不代表外部標準或認證。

2. 適用範圍與用途

- Generic Block/IP -> Subsystem -> Multi-Subsystem -> Full SoC verification。
- 建立或擴充 UVM environment、VIP integration、pattern/sequence、scoreboard、DMA scoreboard、performance、coverage。
- 分析 DUT RTL、PHY、register/programming、VIP behavior、interrupt/DMA/dataflow。
- 自動 build、single simulation、FSDB debug、LSF regression、failure triage、root cause、fix、rerun。
- 將 Requirements / vPlan / Scenario / Pattern / Checker / Coverage / Regression Evidence 串成 traceability。

3. AI 技術與架構

- Graph-Orchestrated Workflow：固定流程骨架與條件 Edge，防止 Agent 任意跳步。
- Plan-and-Execute：先建立工程計畫，再執行；Evidence 改變時局部 replan。
- Multi-Agent：把分析、實作、build、debug、regression、review 分工。
- Blackboard：保存 Current Verification Truth，讓所有 Agent 使用一致狀態。
- Evidence-Grounded ReAct：Reason -> Action -> Observation，但每一輪必須回到工程 Evidence。
- Skill-Governed Execution：Route/Skill Resolver 只載入當前工作所需的能力。
- Verification Memory：保存 verified closure 後的工程知識。
- Autonomous Inference：建立多個 root-cause hypotheses，主動找反證與缺失證據。

4. 整體架構

- Graph 是全局流程骨架；ReAct 是 local decision loop；Memory 是 prior；Blackboard 是 current truth。
- 重要修改前要經過 Multi-Source Evidence Consensus 與 Independent Synthesis。

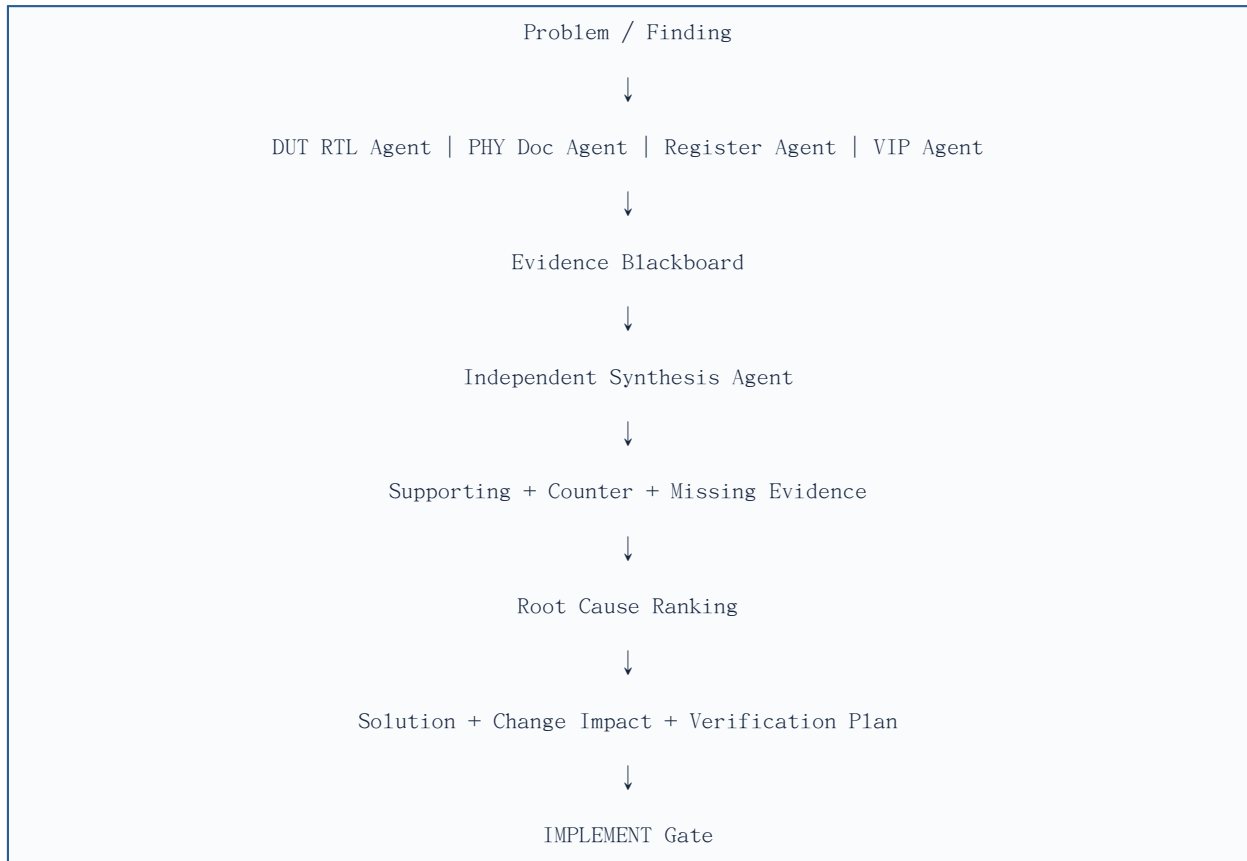


5. Agent 組織

- 7 個 Primary Agents：dv-lead、analysis-agent、implementation-agent、build-agent、debug-agent、regression-agent、review-agent。
- Regression 時每個 submitted JOB_ID 再建立 Dynamic Per-Job Agent Context。
- Job Agent 必須綁定 exact JOB_ID、pattern、options、run_dir、sim.log、Git SHA、Server SHA；不同 job evidence 禁止混用。

6. Multi-Agent Evidence Consensus

- 重要 DUT/VIP/PHY/Register 問題不能由單一 Agent 查完自己下結論。
- DUT Evidence Agent 查 RTL hierarchy/FSM/IRQ/DMA/clock/reset/interface。
- PHY Evidence Agent 查 PHY documents、protocol timing/state、link requirements。
- Register Evidence Agent 查 Programming Guide、register fields、reset value、programming sequence。
- VIP Evidence Agent 查 VIP examples、user manual、VIP source、class reference、sequence/API/config。
- Independent Synthesis Agent 彙整 supporting evidence、counter evidence、conflict、missing evidence、root cause、solution、change impact 與 verification plan。



7. Autonomous Inference

- 先產生多個 competing hypotheses，不接受「看到第一個錯誤就直接改 code」。
- 每個 hypothesis 保存 supporting evidence、counter evidence、missing evidence、confidence 與 next action。
- Next-Best-Action 比較 Information Gain、Root-Cause Discrimination、Verification Value、Runtime/Compute Cost 與 Change Risk。
- Memory 只提高 hypothesis prior；Current Evidence 才能確認本次 root cause。

8. Verification Memory

- Working Memory：當前 Node、Plan、Hypothesis。
- Job Memory：JOB_ID、pattern、log、failure、rerun、result。
- Project Memory：topology、VIP、UVM、build/run、known issues。
- Engineering Memory：closed bug -> root cause -> fix -> verification evidence。
- Organizational Memory：跨 project methodology 與 lessons learned。
- 只有 CLOSED/VERIFIED + single PASS + regression PASS/NOT_REQUIRED + re-audit CLEAN 才能形成可信長期記憶。

9. Generic UVM 架構

- branch-A : DUT-side ports/interfaces 。
- branch_fw : 以 interrupt/event 觸發的 firmware command/control dispatcher 。
- branch-B0/B1/B2... : VIP testing scenarios / patterns / sequences 。
- A 到 B 由 topology 決定，不要求固定 1:1 。
- Shared VIP/resource 僅做 local serialization/arbitration；禁止 global scheduler 將 SoC traffic 全部串行化 。
- Scoreboard、DMA scoreboard、performance calculator、coverage collector 必須支援合法的 port/interface/mode/concurrency 組合 。

10. Simulation 與 Debug

- 統一控制：WAVE=0/1、PA=0/1、FSDB_START、FSDB_STOP、TIMEOUT。
- Single simulation PASS 後，pattern + simulation options + SHA + evidence signature 可自動 promotion 到 regression list。
- Failure path：sim.log -> FSDB/fsdbreport -> Verdi -> DUT/VIP/doc evidence -> hypothesis -> root cause -> fix proposal。

Option	用途
WAVE=0/1	控制 waveform dump
PA=0/1	Project-defined analysis/performance option
FSDB_START	Waveform 起始時間
FSDB_STOP	Waveform 結束時間
TIMEOUT	Simulation timeout

11. LSF Regression

- 所有 submitted jobs 都必須各有獨立 Job Agent。
- Internal monitoring 持續快速檢查 bjobs 與 incremental sim.log；只有 fingerprint 真正變化才立即通知使用者。
- 另外保留固定週期完整 all-job Snapshot。
- LSF DONE != DV PASS；DONE 後仍需分析 completion、UVM_ERROR/UVM_FATAL、assertion、scoreboard、timeout。
- 確認 terminal evidence 後，只允許 bkill exact JOB_ID；其他正常 jobs 必須繼續。

Job	Pattern	LSF	DV Status	Action
103151	usb2_enum p0 HS+p1 HS	RUN	RUNNING	monitor
103152	usb2_enum p0 HS	DONE	ANALYSIS_PENDING	parse sim.log
103162	usb3_gen1 p0+p1	EXIT	ROOT_CAUSE	check exit evidence
103164	usb3_gen1 p0	RUN	RUNNING	monitor

12. Failure -> Fix -> Verify -> Rerun

- Job failure 先 Validate exact JOB_ID/run_dir/sim.log mapping，再保存 evidence。
- debug-agent 與 Multi-Agent evidence tasks 產生 Root Cause、Counter Evidence、Fix Proposal 與 Change Impact。
- Regression 執行期間的新問題先加入 New-Finding Queue；不能立即破壞同一批 baseline。
- 等 ALL jobs final 後集中修正，再建立新 SHA、Build、Verify、Regression。

13. Git / DevOps / SHA

- 標準鏈：git pull/sync -> change -> review -> commit/push -> server exact SHA -> build -> verify -> regression。
- 同一 Regression Batch 必須固定 Git SHA、Server SHA、build identity、config identity。
- 不同 SHA 的 evidence 不可混成同一批 regression 結論。

14. Batch Closure 與 Final Deep Audit

- 啟動 workflow 先深度確認所有 Current Findings，一批修正並 verify。
- Freeze Regression Baseline -> Run ALL Jobs -> New-Finding Queue -> Wait ALL Jobs Final。
- 集中修正所有 New Findings，建立新 baseline，再重跑；直到沒有 actionable finding。
- 最後重新從零深度分析 ALL source files、command.txt、UVM/VIP、branch、scoreboard/DMA/performance/coverage、Make/pattern、Git/DevOps、Claude CLI/Linux environment、VCS/Verdi/FSDB、LSF、requirements/vPlan/scenarios、Memory、Agents、Skills、Graph、Blockboard。
- Final Deep Audit CLEAN 才能標記 SIGNOFF-READY。

15. Human Control Plane

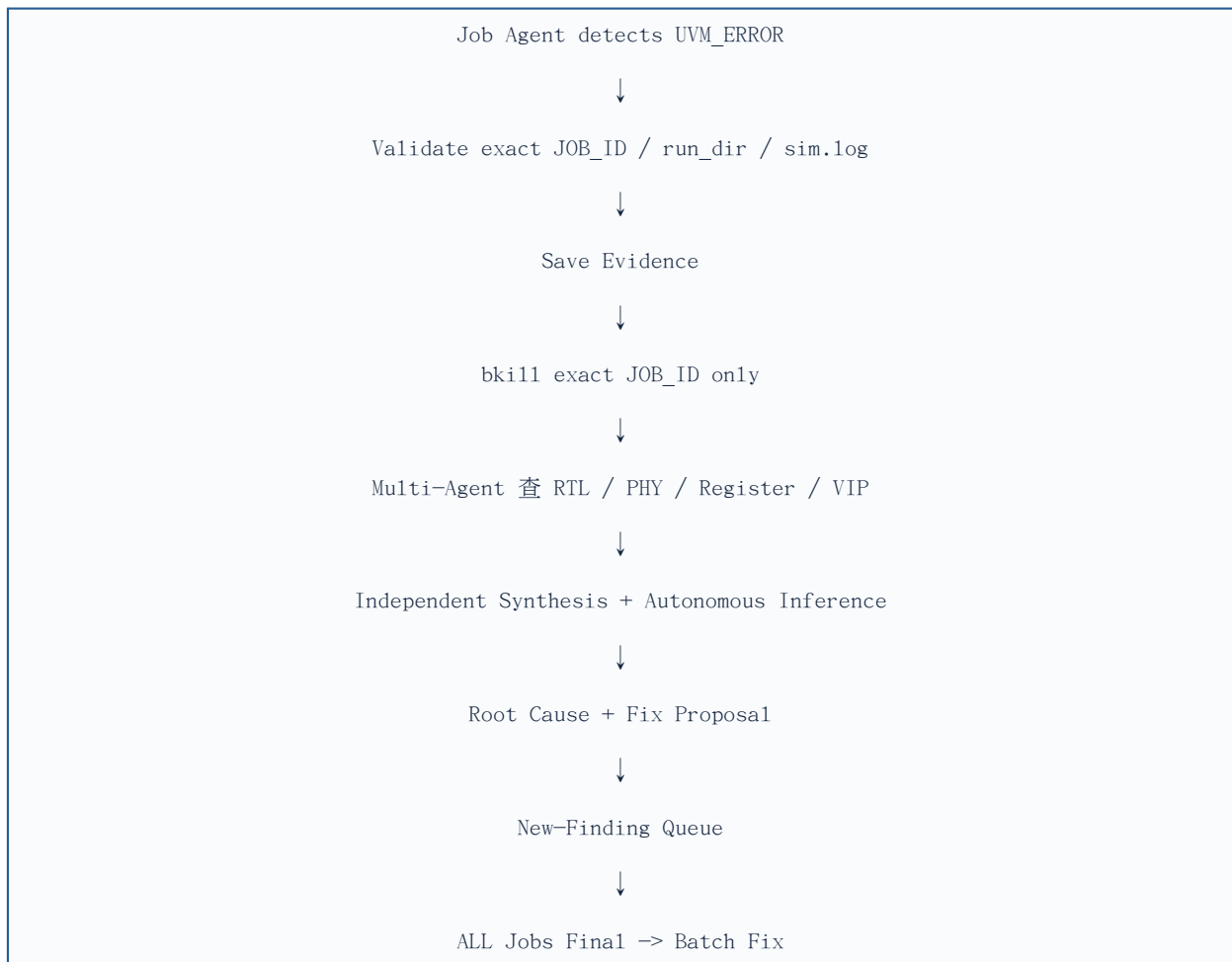
- STATUS：目前 Graph/Plan/Agents/Jobs/Findings/Next Action。
- WHY：顯示工程 reasoning summary。
- EVIDENCE：顯示目前判斷的證據。
- HYPOTHESIS：顯示 hypotheses、反證、缺失 evidence、next-best-action。
- REVIEW：要求獨立 review-agent 挑戰目前結論。
- REDIRECT：保留既有 evidence 後修正方向並局部 replan。
- PAUSE / RESUME / STOP / TAKEOVER：工程師永遠保留控制權。

16. 詳細使用範例

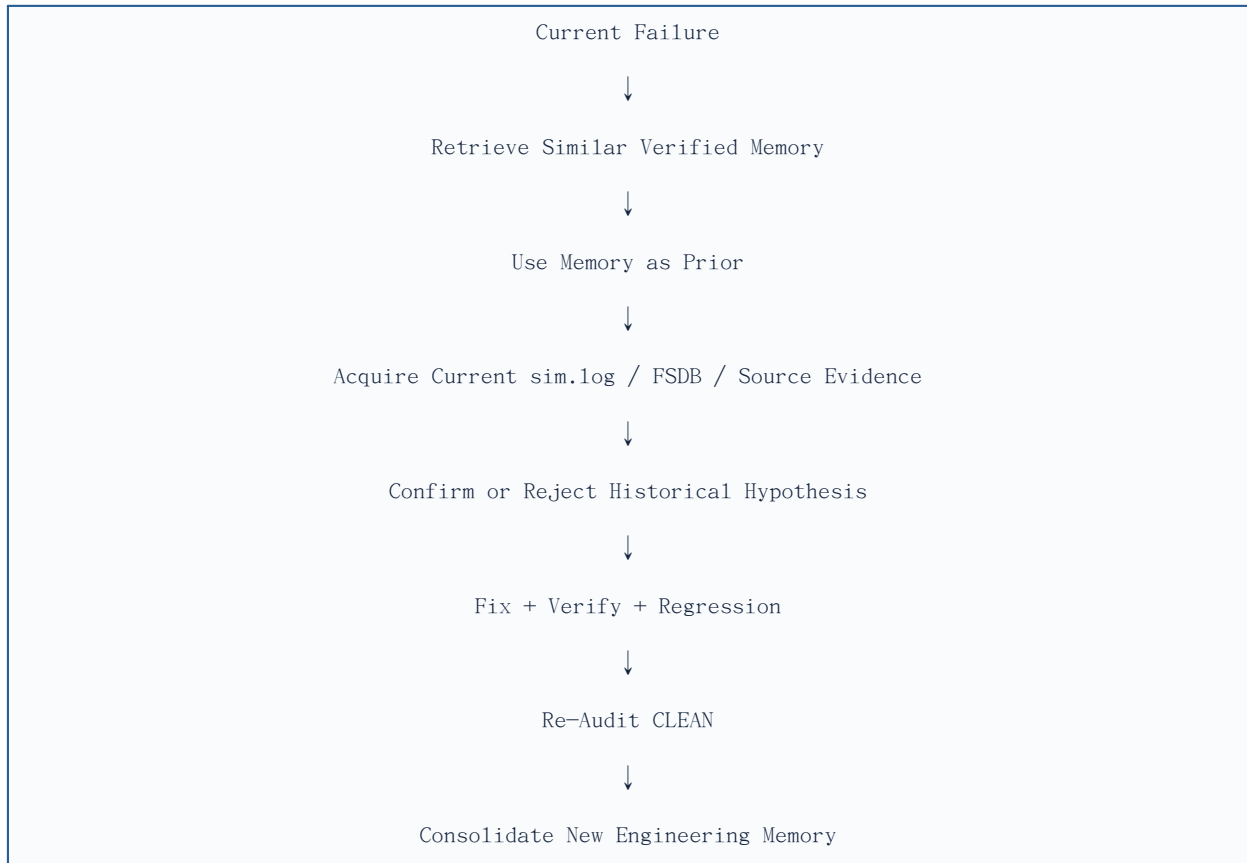
範例 A – 新增 Pattern 並加入 Regression



範例 B – Job 出現 UVM_ERROR



範例 C – Memory-Assisted Debug



17. Signoff Gate

Gate	最低要求
Requirements	Critical/High requirement 有 traceability 與 evidence
Findings	Open actionable critical = 0
Evidence Consensus	重要問題完成 Multi-Source cross-check
Git / Server	Exact SHA MATCH
Build / Single Verify	PASS
Wave Evidence	代表性 WAVE/FSDB evidence PASS
Regression	Targeted + Dependency + Safety closure
Memory	只保存 verified closure
Final Deep Audit	CLEAN
Harness State	SIGNOFF-READY ; 最終 signoff 依公司/專案 policy

工具定位：DV Agent Harness L5 位於 simulator/debugger 之上的 orchestration 與 reasoning 層；VCS、Verdi、FSDB、LSF、Git、Make 等仍是實際執行與 evidence 來源。Harness 的責任是讓 AI 使用這些工具時具有流程、證據、版本一致性、記憶、推論、Review 與 Closure discipline。